PDVix CRM — Documentação Técnica Completa

Versão: 3.0.0-draft | **Data:** 2026-03-06 | **Autor:** Gerado via análise de código

---

Sumário

1. Visão Geral da Arquitetura
2. Estado Atual do Sistema
3. Schema do Banco de Dados — Servidor
4. Schema do Banco Local — PDV SQLite
5. API REST — Rotas Existentes
6. Protocolo de Sincronização PDV → Servidor
7. WebSocket Server — PDVix Gateway
8. Novos Módulos — Design e Especificação
   - 8.1 Multi-Lojas
   - 8.2 Módulo PDV (Configuração e Comandos Remotos)
   - 8.3 Pagar.me — PIX e Stone POS
   - 8.4 Dashboard de Vendas
   - 8.5 Cancelamentos
   - 8.6 Espelho PDV
   - 8.7 Comandas (Pré-Venda)
9. Migrations Necessárias
10. Pendências no Banco SQLite do PDV
11. Checklist de Implementação

---

1. Visão Geral da Arquitetura

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    SERVIDOR (XAMPP / Linux)                                  │
│                                                                              │
│  ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐  │
│  │  PHP (Apache)        │  │  Workerman WS        │  │  Redis 6379          │  │
│  │  /api/*              │  │  :8443 (WSS)         │  │  filas / estado      │  │
│  │  MVC + REST          │◄─┤  pdv_server.php      │◄─┤  pdv:cmd:{id}        │  │
│  └──────────────────────┘  └──────────────────────┘  └──────────────────────┘  │
│          │                          │                          │
```

```
│  ┌──────────▼──────────┐          │ publica                  │
│  │ MariaDB 10.4        │          │ consome                  │
│  │ pdvix_crm           │          │                          │
│  └─────────────────────┘          │                          │
└──────────────────────────────────┼──────────────────────────────
                                   │ WSS / HTTPS
                  ┌────────────────┼────────────────┐
                  │                │                │
        ┌─────────▼─────────┐  ┌─────────▼─────────┐  ┌─────────▼─────────┐
        │ PDV Electron      │  │ PDV Electron      │  │ Painel Admin      │
        │ Loja 1/PDV1       │  │ Loja 1/PDV2       │  │ AdminLTE SPA      │
        │ SQLite local      │  │ SQLite local      │  │ Browser           │
        └───────────────────┘  └───────────────────┘  └───────────────────┘
```

Fluxo de dados principal

```
PDV (offline) → SQLite local → [reconecta] → POST /api/pdv/sync-venda → MariaDB
PDV (online)  → WSS pdv_server.php → Redis → fila pdv:cmd:{loja}:{pdv} → PDV executa
Admin         → Painel SPA → REST API → MariaDB
Admin         → WebSocket subscribe → recebe eventos em tempo real do PDV
Pagar.me      → Webhook POST /api/webhook/pagarme → Redis → WSS → PDV finaliza venda
```

---

2. Estado Atual do Sistema

Módulos implementados ✅

Módulo	Controller	Model	Service	Rotas	Front-e
Auth	AuthController	UserModel	RedisService	`/auth`, `/logout`	login.ht
Usuários	UserController	UserModel	—	`/api/users`	usuarios
Produtos	ProdutoController	ProdutoModel	ProdutoService	`/api/produtos`	produto
Estoque	EstoqueController	EstoqueModel	EstoqueService	`/api/estoque`	estoque.
Movimentações	MovimentacaoController	MovimentacaoModel	—	`/api/movimentacoes`	movime
Vendas	VendaController	VendaModel	VendaService	`/api/vendas`	vendas.l
Pagamentos	PagamentoController	PagamentoModel	PagamentoService	`/api/pagamentos`	pagame
Caixas (leitura)	PdvCaixaController	—	—	`/api/pdv/caixas`	caixa.ht

Módulo	Controller	Model	Service	Rotas	Front-e
Sync Venda PDV	PdvSyncController	—	—	`/api/pdv/sync-venda`	—
Sync Caixa PDV	PdvCaixaController	—	—	`/api/pdv/sync-caixa`	—
Carga Inicial PDV	CargaInicialController	—	—	`/api/carga-inicial`	—

Módulos a criar ⬜

Módulo	Prioridade	Depende de
Multi-Lojas	Alta	— (base para os demais)
PDV Config/Comandos	Alta	Multi-Lojas + WebSocket
WebSocket Server	Alta	Redis
Pagar.me (PIX)	Alta	WebSocket
Pagar.me (Stone POS)	Média	WebSocket
Cancelamentos	Alta	Vendas + WebSocket
Comandas	Média	Vendas
Dashboard	Média	Vendas + Caixas
Espelho PDV	Baixa	Caixas

---

3. Schema do Banco de Dados — Servidor

Tabelas existentes

`usuarios`

```sql
id, login (UNIQUE), password (bcrypt), perfil ENUM('operador','gerente','administrador'),
nome, cpf (UNIQUE via código), email, telefone,
status ENUM('ativado','desativado'), data_criacao, criado_por, ultimo_login
```

produtos

```sql
id, nome, codigo_interno_alternativo (INT),
preco_venda DECIMAL(10,2), custo_item DECIMAL(10,2),
fator_embalagem INT DEFAULT 1,
unidade_base ENUM('UN','G') DEFAULT 'UN',
fornecedor_id FK(fornecedores), ultima_alteracao, ultima_alteracao_por,
bloqueado INT DEFAULT 0
```

> **Nota:** Não há FK explícita de `produtos` → `lojas` ainda. Será adicionada na migration v3.

produtos_codigos_barras

```sql
id, produto_id FK(produtos), codigo_barras VARCHAR(50),
tipo_embalagem ENUM('UN','CX','KG','G'),
preco_venda DECIMAL(10,2)
UNIQUE(produto_id, tipo_embalagem)
```

estoque

```sql
id, produto_id FK(produtos), quantidade_atual INT, data_atualizacao
```

> **Nota:** Uma linha por produto. `atualizarQuantidade()` faz UPDATE se existe, INSERT se não existe.

estoque_movimentacoes

```sql
id, produto_id FK(produtos),
tipo_movimento ENUM('ENTRADA','SAIDA','AJUSTE'),
quantidade DECIMAL(10,3),          -- unidade original (pode ser 1.5 KG)
unidade_origem ENUM('UN','CX','KG','G'),
codigo_barras_usado VARCHAR(50),
motivo VARCHAR(100), referencia_id INT, data_movimento,
usuario_id INT, origem ENUM('VENDA','COMPRA','AJUSTE','DEVOLUCAO','ESTORNO')
```

vendas

```sql
```

```
id, numero_venda VARCHAR(30) UNIQUE,
data_venda DATETIME, cliente_id, usuario_id FK(usuarios),
subtotal, desconto, acrescimo, total DECIMAL(10,2),
status ENUM('aberta','finalizada','cancelada'),
observacao TEXT, created_at, updated_at,
numero_pdv VARCHAR(10) DEFAULT '01'
```

> **Nota multi-loja:** Adicionar `loja_id FK(lojas)` na migration v3.

venda_itens

```sql
id, venda_id FK(vendas), produto_id FK(produtos),
quantidade DECIMAL(10,3), valor_unitario DECIMAL(10,2),
subtotal DECIMAL(10,2), created_at
```

pagamentos_venda

```sql
id, venda_id FK(vendas),
tipo_pagamento ENUM('pix','convenio','pos_debito','pos_credito','pos_pix','dinheiro','outros'),
valor DECIMAL(10,2),
referencia_externa VARCHAR(100),   -- order_id do pagar.me
descricao VARCHAR(150),
status ENUM('pendente','confirmado','cancelado'),
gerado_por INT,                    -- usuario_id que gerou
created_at
```

> **Nota pagar.me:** `referencia_externa` receberá o `order.id` do pagar.me. Adicionar `pagarme_charge_id` e `pagarme_status` na migration v3.

caixa_sessoes

```sql
id, numero_pdv VARCHAR(10),
usuario_id FK(usuarios), abertura_em, fechamento_em,
valor_abertura, total_dinheiro, total_pix, total_debito,
total_credito, total_convenio, total_outros DECIMAL(10,2),
total_vendas INT, total_canceladas INT, total_sangrias,
saldo_esperado, caixa_contado, diferenca DECIMAL(10,2),
status ENUM('aberto','fechado'), observacao,
sincronizado_em, created_at, updated_at
```

> **Nota multi-loja:** Adicionar `loja_id FK(lojas)` na migration v3.

caixa_sangrias

```sql
id, caixa_sessao_id FK(caixa_sessoes) ON DELETE CASCADE,
usuario_id FK(usuarios), valor DECIMAL(10,2),
motivo VARCHAR(255), data_hora, created_at
```

supervisores_cartoes

```sql
id, usuario_id FK(usuarios), codigo_cartao VARCHAR(100) UNIQUE,
descricao, permite_desconto_item TINYINT, permite_desconto_venda TINYINT,
permite_cancelar_item TINYINT, permite_cancelar_venda TINYINT, ativo TINYINT
```

clientes

```sql
id, nome VARCHAR(160), cpf VARCHAR(14) UNIQUE,
telefone VARCHAR(20), status ENUM('ativo','inativo'), criado_em
```

config

```sql
chave VARCHAR(80) PK, valor TEXT
-- Registros atuais: api_token, versao
```

fornecedores

```sql
id, cnpj UNIQUE, razao_social, nome_fantasia,
endereco, telefone, ativo, insc_municipal, cidade, estado
```

historico

```sql
id, data DATE, usuario VARCHAR(60), tipo_atividade,
log TEXT, ip VARCHAR(60), quando DATETIME
```

---

4. Schema do Banco Local — PDV SQLite

Comparação Servidor × PDV

Tabela	Servidor	PDV SQLite	Diferença
usuarios	id, login, password, perfil, nome, cpf, status	id, login, **password_local**, perfil, nome, cpf, status	Campo renomeado; sem email/telefone no PDV
produtos	+ custo_item, fornecedor_id, ultima_alteracao_por, bloqueado	+ atualizado_em	PDV é read-only; campos de gestão omitidos
produtos_codigos_barras	imagem_path	—	PDV não usa imagem
vendas	+ numero_pdv, cliente_id, created_at, updated_at	+ **caixa_sessao_id**, cliente_cpf, cliente_nome, **sincronizado**, **id_servidor**	PDV tem campos de controle de sync
venda_itens	—	+ produto_nome, desconto_item, codigo_barras_usado, unidade_origem	PDV armazena snapshot do produto
pagamentos_venda	+ referencia_externa, descricao, gerado_por	+ **sincronizado**, data_hora	PDV tem controle de sync
caixa_sessoes	+ numero_pdv, total_sangrias, saldo_esperado, caixa_contado, diferenca, sincronizado_em	sem esses campos	Servidor acumula; PDV é simples
sangrias (PDV) / caixa_sangrias (Server)	caixa_sessao_id	caixa_sessao_id + **sincronizado**	PDV tem controle de sync
clientes	—	—	Idênticas
supervisores_cartoes	—	—	Idênticas
config	chave, valor	chave, valor	Idênticas

Campos de controle de sync (apenas no SQLite)

- vendas.sincronizado — 0=pendente, 1=sincronizado
- vendas.id_servidor — ID atribuído após sync

- `pagamentos_venda.sincronizado` — idem
- `sangrias.sincronizado` — idem

> **Regra de ouro:** O PDV nunca modifica um registro já sincronizado (`sincronizado = 1`). Em caso de reenvio, o servidor é idempotente via `numero_venda`.

---

5. API REST — Rotas Existentes

Autenticação

Método	Rota	Controller	Auth	Descrição
GET	`/login`	AuthController::index	Pública	Tela de login
POST	`/auth`	AuthController::authentication	Pública	Autentica, retorna sessão PHP
GET/POST	`/logout`	AuthController::logout	Sessão	Encerra sessão

PDV Electron — Token estático

Todas essas rotas usam `?token=XXX` (chave `api_token` na tabela `config`). Não requerem sessão PHP — ideal para Electron sem cookie.

Método	Rota	Controller	Descrição
GET	`/api/carga-inicial`	CargaInicialController::index	Retorna produtos, códigos de barras, usuários, cartões supervisores, clientes
POST	`/api/pdv/sync-venda`	PdvSyncController::sync	Sincroniza venda offline (idempotente via `numero_venda`)
POST	`/api/pdv/sync-caixa`	PdvCaixaController::sync	Sincroniza sessão de caixa offline
GET	`/api/pdv/caixas`	PdvCaixaController::listar	Lista caixas (DataTables, sessão PHP)
GET	`/api/pdv/caixas/detalhe`	PdvCaixaController::detalhe	Detalhe de uma sessão + sangrias

Usuários

Método	Rota	Perfil mínimo	Descrição
GET	`/api/users`	gerente	Listar (DataTables server-side)
POST	`/api/users`	gerente	Criar usuário

Método	Rota	Perfil mínimo	Descrição
PUT	/api/users	gerente	Atualizar usuário
DELETE	/api/users	administrador	Excluir usuário
PATCH	/api/users/status	gerente	Toggle ativado/desativado

Produtos

Método	Rota	Perfil mínimo	Descrição
GET	/api/produtos	operador	Listar (DataTables)
GET	/api/produtos?simples=1	operador	Lista simples para selects
POST	/api/produtos	gerente	Criar produto
PUT	/api/produtos	gerente	Atualizar produto
DELETE	/api/produtos	administrador	Excluir (bloqueia se tem movimentações)
PATCH	/api/produtos/status	gerente	Toggle bloqueado
GET	/api/produtos/historico?id=X	operador	Histórico de movimentações do produto

Estoque

Método	Rota	Perfil mínimo	Descrição
GET	/api/estoque	operador	Listar posição de estoque (DataTables)
POST	/api/estoque/movimentar	operador	Registrar entrada/saída/ajuste

Movimentações

Método	Rota	Perfil mínimo	Descrição
GET	/api/movimentacoes	operador	Listar histórico completo (DataTables, somente leitura)

Vendas

Método	Rota	Perfil mínimo	Descrição
GET	/api/vendas	operador	Listar (DataTables)

Método	Rota	Perfil mínimo	Descrição
POST	/api/vendas	operador	Criar venda com itens (baixa estoque)
DELETE	/api/vendas	gerente	Excluir venda (só status=aberta)
PATCH	/api/vendas/status	gerente	Alterar status (cancelar, etc.)
GET	/api/vendas/itens?id=X	operador	Itens + pagamentos de uma venda
POST	/api/vendas/finalizar	operador	Finalizar venda (verifica pagamentos)

Pagamentos

Método	Rota	Perfil mínimo	Descrição
GET	/api/pagamentos	operador	Listar (DataTables)
POST	/api/pagamentos	operador	Criar pagamento
PUT	/api/pagamentos	gerente	Atualizar pagamento
DELETE	/api/pagamentos	gerente	Excluir (só status=pendente)
PATCH	/api/pagamentos/status	gerente	Alterar status

---

6. Protocolo de Sincronização PDV → Servidor

6.1 Carga Inicial (boot do PDV)

Quando: Na abertura do PDV ou ao detectar conexão com o servidor.

```
GET /api/carga-inicial?token={api_token}
```

Resposta:

```
json
```

```
{
  "status": "success",
  "data": {
    "produtos":          [...],  // id, nome, codigo_interno_alternativo, preco_venda, fator_embalagem, unidade_base, bloqueado
    "codigos_barras":     [...],  // produto_id, codigo_barras, tipo_embalagem, preco_venda
    "usuarios":          [...],  // id, login, perfil, nome, cpf, status  (sem password)
    "supervisores_cartoes": [...],  // id, usuario_id, codigo_cartao, permissões...
    "clientes":          [...]   // id, nome, cpf, telefone, status
  }
}
```

O PDV deve:

1. Truncar tabelas locais `produtos`, `produtos_codigos_barras`, `usuarios`, `supervisores_cartoes`, `clientes`
2. Reinserir todos os registros recebidos
3. **NÃO** truncar `vendas`, `venda_itens`, `pagamentos_venda`, `caixa_sessoes`, `sangrias` — esses têm dados locais pendentes de sync

6.2 Sincronização de Venda

Quando: Ao finalizar cada venda (online) ou em lote ao reconectar (offline).

```
POST /api/pdv/sync-venda?token={api_token}
Content-Type: application/json
```

Payload:

```
json
```

```
{
  "numero_venda": "PDV01-1234567890",
  "usuario_id": 1,
  "cliente_id": null,
  "cliente_cpf": "00000000000",
  "cliente_nome": "CONSUMIDOR FINAL",
  "subtotal": 10.00,
  "desconto": 0.00,
  "acrescimo": 0.00,
  "total": 10.00,
  "data_venda": "2026-03-05 00:19:42",
  "observacao": null,
  "itens": [
    {
      "produto_id": 1,
      "produto_nome": "Biscoito",
      "quantidade": 2,
      "valor_unitario": 5.00,
      "desconto_item": 0,
      "subtotal": 10.00,
      "codigo_barras_usado": "7891234567890",
      "unidade_origem": "UN"
    }
  ],
  "pagamentos": [
    {
      "tipo_pagamento": "dinheiro",
      "valor": 10.00,
      "referencia_externa": null
    }
  ]
}
```


Resposta de sucesso:

```
json

{ "status": "success", "data": { "id_servidor": 42 }, "message": "Venda sincronizada com sucesso." }
```

Resposta idempotente (já existia):

```
json

{ "status": "success", "data": { "id_servidor": 42 }, "message": "Venda já estava sincronizada — nenhuma ação necessária." }
```

O PDV deve após sync bem-sucedido:

```sql
UPDATE vendas SET sincronizado = 1, id_servidor = 42 WHERE numero_venda = 'PDV01-1234567890';
UPDATE pagamentos_venda SET sincronizado = 1 WHERE venda_id = (local_id);
```

6.3 Sincronização de Caixa

Quando: Ao fechar a sessão de caixa (online) ou ao reconectar.

```
POST /api/pdv/sync-caixa?token={api_token}
Content-Type: application/json
```

Payload:

```json
{
  "numero_pdv":      "01",
  "usuario_id":      1,
  "abertura_em":     "2026-03-05 08:00:00",
  "fechamento_em":   "2026-03-05 18:00:00",
  "valor_abertura":  100.00,
  "total_dinheiro":  500.00,
  "total_pix":       200.00,
  "total_debito":    0.00,
  "total_credito":   0.00,
  "total_convenio":  0.00,
  "total_outros":    0.00,
  "total_vendas":    15,
  "total_canceladas": 1,
  "total_sangrias":  50.00,
  "saldo_esperado":  550.00,
  "caixa_contado":   null,
  "status":          "fechado",
  "observacao":      null,
  "sangrias": [
    {
      "usuario_id": 1,
      "valor":      50.00,
      "motivo":     "Pagamento de fornecedor",
      "data_hora":  "2026-03-05 14:30:00"
    }
  ]
}
```

Chave de idempotência: `(numero_pdv + usuario_id + abertura_em)`. Reenvios atualizam os totais sem duplicar o caixa.

6.4 Ordem de sync ao reconectar

O PDV deve sincronizar nesta ordem para manter integridade referencial:

```
1. GET /api/carga-inicial            → atualiza catálogo
2. POST /api/pdv/sync-venda (loop)   → vendas pendentes (sincronizado=0), uma por vez
3. POST /api/pdv/sync-caixa          → caixa da sessão atual ou pendente
4. WS connect + authenticate         → entra online no WebSocket
```

6.5 Novos campos a sincronizar (futuro — ver seção 10)

Com a implementação de multi-loja e cancelamentos, o payload de sync-venda receberá:

- `loja_id` — obrigatório quando multi-loja ativo
- `cancelamentos` — array de itens ou venda cancelada no PDV
- `numero_pdv` — já existe em `vendas` no servidor, mas falta no payload atual

---

7. WebSocket Server — PDVix Gateway

7.1 Visão Geral

O servidor WebSocket (`pdv_server.php`) usa **Workerman** com SSL e Redis para:

- Receber status/heartbeat dos PDVs
- Enviar comandos remotos do admin para PDVs específicos
- Distribuir confirmações de pagamento (pagar.me webhook → Redis → WS → PDV)
- Notificar o painel admin em tempo real

7.2 Estrutura de canais

```
pdv:{loja_id}:{numero_pdv}   — canal privado por PDV
admin:loja:{loja_id}         — admin da loja recebe eventos de todos os PDVs da loja
admin:global                 — superadmin recebe tudo
```

7.3 Filas Redis consumidas pelo servidor WS

Chave Redis	Produtor	Consumidor	Conteúdo
`pdv:cmd:{loja_id}:{numero_pdv}`	API REST (admin)	WS → PDV	Comandos remotos
`pdv:pagamento:{numero_venda}`	Webhook pagar.me	WS → PDV	Confirmação de pagamento

Chave Redis	Produtor	Consumidor	Conteúdo
`pdv:carga:{loja_id}:{numero_pdv}`	API REST	WS → PDV	Carga diferencial
`admin:evento:{loja_id}`	PDV via WS	WS → Admin	Eventos de status, venda, etc.

7.4 Protocolo de mensagens (JSON)

Toda mensagem WS tem o envelope:

```
{ "event": "nome_do_evento", "payload": {}, "meta": { "ts": 1234567890 } }
```

Eventos PDV → Servidor

Event	Payload	Descrição
`pdv:auth`	`{ token, loja_id, numero_pdv, usuario_id }`	Autenticação do PDV no WS. Token = api_token
`pdv:heartbeat`	`{ loja_id, numero_pdv, status, caixa_aberto, ultima_venda_em }`	Heartbeat a cada 30s
`pdv:venda_finalizada`	`{ numero_venda, total, tipo_pagamento }`	Notifica admin que venda foi finalizada
`pdv:caixa_aberto`	`{ numero_pdv, usuario_id, abertura_em }`	Caixa foi aberto
`pdv:caixa_fechado`	`{ numero_pdv, total_geral }`	Caixa foi fechado
`pdv:cmd_resultado`	`{ cmd_id, sucesso, mensagem }`	Resultado de um comando remoto

Eventos Servidor → PDV

Event	Payload	Descrição
ws:auth_ok	{ numero_pdv, loja_id }	Autenticação aceita
ws:auth_fail	{ motivo }	Autenticação rejeitada
pdv:comando	{ cmd_id, tipo, dados }	Comando remoto (ver tipos abaixo)
pdv:pagamento_confirmado	{ numero_venda, valor, tipo, referencia }	Pagamento confirmado no pagar.me
pdv:pagamento_cancelado	{ numero_venda, motivo }	Pagamento falhou/cancelado
pdv:carga_disponivel	`{ tipo: 'completa'	'diferencial', versao }`

Tipos de comando remoto (pdv:comando)

tipo	dados	Ação no PDV
reiniciar	{}	Reinicia o app Electron
desligar	{}	Fecha o app Electron
fechar_caixa	{ motivo }	Força fechamento do caixa
enviar_carga	`{ tipo: 'completa'	'diferencial' }`
cancelar_item	{ venda_id_local, produto_id, quantidade }	Cancela item de venda aberta
cancelar_venda	{ venda_id_local, motivo }	Cancela venda inteira
desconto_item	{ venda_id_local, produto_id, percentual }	Aplica desconto em item
desconto_venda	{ venda_id_local, percentual }	Aplica desconto na venda
finalizar_venda	{ venda_id_local, pagamentos[] }	Admin finaliza venda remotamente
enviar_comanda	{ comanda_id, itens[], cliente }	Envia pré-venda para o PDV

Eventos Servidor → Admin (painel)

Event	Payload	Descrição
`admin:pdv_online`	`{ loja_id, numero_pdv, usuario_nome }`	PDV conectou
`admin:pdv_offline`	`{ loja_id, numero_pdv }`	PDV desconectou
`admin:pdv_status`	`{ loja_id, numero_pdv, ...heartbeat }`	Atualização de status
`admin:venda_nova`	`{ loja_id, numero_pdv, numero_venda, total }`	Nova venda finalizada
`admin:pagamento_confirmado`	`{ numero_venda, valor, tipo }`	Pagamento confirmado

7.5 Arquivo `pdv_server.php` — Estrutura completa

```php
```

```php
<?php
/**
 * PDVix WebSocket Gateway
 *
 * Responsabilidades:
 *   - Autenticar PDVs e admins via token
 *   - Gerenciar canais por loja/PDV
 *   - Consumir filas Redis e distribuir mensagens
 *   - Receber heartbeats e detectar PDVs offline
 *   - Rotear comandos remotos admin → PDV específico
 *
 * Execução:
 *   php pdv_server.php start -d   (daemon)
 *   php pdv_server.php stop
 *   php pdv_server.php restart
 *   php pdv_server.php status
 *
 * Dependências (composer):
 *   workerman/workerman: ^4.1
 *   predis/predis: ^2.0      (ou extensão Redis nativa)
 *
 * Porta padrão: 8443 (WSS com SSL)
 * PID: /tmp/pdvix_ws.pid
 * Log: /var/www/html/pdvix_ws.log
 */

require __DIR__ . '/vendor/autoload.php';
require __DIR__ . '/../config.php';  // define DB_HOST, DB_NAME, REDIS_HOST, API_TOKEN_CACHE_TTL

use Workerman\Worker;
use Workerman\Timer;
use Workerman\Connection\TcpConnection;

date_default_timezone_set('America/Cuiaba');

// ─── SSL ──────────────────────────────────────────────────────────────────
$SSL_CONTEXT = [
    'ssl' => [
        'local_cert'        => '/etc/letsencrypt/live/seu-dominio.com/fullchain.pem',
        'local_pk'          => '/etc/letsencrypt/live/seu-dominio.com/privkey.pem',
        'verify_peer'       => false,
        'verify_peer_name'  => false,
    ]
];

// ─── Worker ───────────────────────────────────────────────────────────────
```

```php
$ws = new Worker("websocket://0.0.0.0:8443", $SSL_CONTEXT);
$ws->transport = 'ssl';
$ws->count     = 1;   // Single-process — compartilha $clients/$channels sem lock

// ─── Estado em memória ──────────────────────────────────────────────────────
/**
 * $clients[ connection_id ] = [
 *   'conn'       => TcpConnection,
 *   'tipo'       => 'pdv' | 'admin' | null,
 *   'loja_id'    => int|null,
 *   'numero_pdv' => string|null,
 *   'usuario_id' => int|null,
 *   'auth'       => bool,
 *   'last_ping'  => int (timestamp),
 * ]
 */
$clients = [];

/**
 * Índice para lookup rápido: pdv_key → connection_id
 * pdv_key = "{loja_id}:{numero_pdv}"
 */
$pdvIndex = [];

// ─── Conexão ────────────────────────────────────────────────────────────────
$ws->onConnect = function(TcpConnection $conn) use (&$clients) {
    $clients[$conn->id] = [
        'conn'       => $conn,
        'tipo'       => null,
        'loja_id'    => null,
        'numero_pdv' => null,
        'usuario_id' => null,
        'auth'       => false,
        'last_ping'  => time(),
    ];

    // Aguarda autenticação por 10s antes de fechar por timeout
    $conn->authTimer = Timer::add(10, function() use ($conn, &$clients) {
        if (!$clients[$conn->id]['auth'] ?? false) {
            $conn->send(json_encode(['event' => 'ws:auth_fail', 'payload' => ['motivo' => 'Timeout de autenticação']]));
            $conn->close();
        }
    }, null, false); // one-shot
};

// ─── Desconexão ─────────────────────────────────────────────────────────────
$ws->onClose = function(TcpConnection $conn) use (&$clients, &$pdvIndex) {
```

```
    $client = $clients[$conn->id] ?? null;

    if ($client && $client['tipo'] === 'pdv' && $client['auth']) {
        $pdvKey = "{$client['loja_id']}:{$client['numero_pdv']}";
        unset($pdvIndex[$pdvKey]);

        // Notifica admins da loja que PDV saiu
        _broadcastAdmin($clients, $client['loja_id'], [
            'event'   => 'admin:pdv_offline',
            'payload' => [
                'loja_id'    => $client['loja_id'],
                'numero_pdv' => $client['numero_pdv'],
            ],
        ]);

        echo "[OFFLINE] PDV {$pdvKey}\n";
    }

    unset($clients[$conn->id]);
};

// ─── Mensagens ──────────────────────────────────────────────────────────────
$ws->onMessage = function(TcpConnection $conn, string $raw) use (&$clients, &$pdvIndex) {
    $msg = json_decode($raw, true);
    if (!$msg || empty($msg['event'])) return;

    $event   = $msg['event'];
    $payload = $msg['payload'] ?? [];
    $client  = &$clients[$conn->id];

    // ── Autenticação (obrigatório antes de tudo) ──────────────────────────
    if ($event === 'pdv:auth') {
        _autenticarPdv($conn, $client, $payload, $pdvIndex, $clients);
        return;
    }

    if ($event === 'admin:auth') {
        _autenticarAdmin($conn, $client, $payload);
        return;
    }

    // ── Rejeita não-autenticados ──────────────────────────────────────────
    if (!$client['auth']) {
        $conn->send(json_encode(['event' => 'ws:error', 'payload' => ['motivo' => 'Não autenticado']]));
        return;
    }
```

```
$client['last_ping'] = time();

// ── Roteamento por tipo de cliente ──────────────────────────────────────
switch ($event) {

    // PDV → Servidor
    case 'pdv:heartbeat':
        $client['last_ping'] = time();
        _broadcastAdmin($clients, $client['loja_id'], [
            'event'   => 'admin:pdv_status',
            'payload' => array_merge($payload, [
                'loja_id'    => $client['loja_id'],
                'numero_pdv' => $client['numero_pdv'],
            ]),
        ]);
        break;

    case 'pdv:venda_finalizada':
        _broadcastAdmin($clients, $client['loja_id'], [
            'event'   => 'admin:venda_nova',
            'payload' => array_merge($payload, [
                'loja_id'    => $client['loja_id'],
                'numero_pdv' => $client['numero_pdv'],
            ]),
        ]);
        break;

    case 'pdv:caixa_aberto':
    case 'pdv:caixa_fechado':
        _broadcastAdmin($clients, $client['loja_id'], [
            'event'   => $event,
            'payload' => array_merge($payload, ['numero_pdv' => $client['numero_pdv']]),
        ]);
        break;

    case 'pdv:cmd_resultado':
        // Resultado de comando — loga e notifica admin
        echo "[CMD_RESULT] PDV {$client['loja_id']}:{$client['numero_pdv']} → "
            . json_encode($payload) . "\n";
        _broadcastAdmin($clients, $client['loja_id'], [
            'event'   => 'admin:cmd_resultado',
            'payload' => array_merge($payload, ['numero_pdv' => $client['numero_pdv']]),
        ]);
        break;

    // Admin → PDV (via WS direto, alternativa à fila Redis)
    case 'admin:enviar_comando':
```

```
        if ($client['tipo'] !== 'admin') break;
        $targetKey = ($payload['loja_id'] ?? '') . ':' . ($payload['numero_pdv'] ?? '');
        if (isset($pdvIndex[$targetKey])) {
            $targetConnId = $pdvIndex[$targetKey];
            $clients[$targetConnId]['conn']->send(json_encode([
                'event'   => 'pdv:comando',
                'payload' => $payload,
            ]));
        } else {
            $conn->send(json_encode([
                'event'   => 'admin:erro',
                'payload' => ['motivo' => "PDV {$targetKey} não está conectado"],
            ]));
        }
        break;
    }
};

// ——— Worker Start — Redis consumers ———————————————————————————————————————
$ws->onWorkerStart = function() use (&$clients, &$pdvIndex) {
    $redis = new Redis();
    $redis->connect(REDIS_HOST ?? '127.0.0.1', 6379);
    echo "[REDIS] Conectado\n";

    /**
     * Fila de comandos remotos: admin enfileira via API REST,
     * WS distribui para o PDV correto.
     *
     * Chave: pdv:cmd:{loja_id}:{numero_pdv}
     * A API REST enfileira com: $redis->rPush("pdv:cmd:{$lojaId}:{$numeroPdv}", json_encode($cmd))
     */
    Timer::add(0.1, function() use ($redis, &$clients, &$pdvIndex) {
        foreach ($pdvIndex as $pdvKey => $connId) {
            $msg = $redis->lPop("pdv:cmd:{$pdvKey}");
            if (!$msg) continue;
            if (!isset($clients[$connId])) continue;
            $clients[$connId]['conn']->send($msg);
            echo "[CMD] → PDV {$pdvKey}\n";
        }
    });

    /**
     * Fila de confirmações de pagamento.
     * O webhook do pagar.me (API REST) enfileira:
     *   $redis->rPush("pdv:pagamento:{$numeroVenda}", json_encode($payload))
     * O WS precisa saber qual PDV tem a venda. Usa lookup via Redis:
     *   chave: pdv:venda_pdv:{numero_venda} → "{loja_id}:{numero_pdv}"
```

```
    */
    Timer::add(0.2, function() use ($redis, &$clients, &$pdvIndex) {
        $msg = $redis->lPop('pdv:fila_pagamentos');
        if (!$msg) return;

        $data = json_decode($msg, true);
        if (!$data || empty($data['numero_venda'])) return;

        $pdvKey = $redis->get("pdv:venda_pdv:{$data['numero_venda']}");
        if (!$pdvKey || !isset($pdvIndex[$pdvKey])) return;

        $connId = $pdvIndex[$pdvKey];
        if (!isset($clients[$connId])) return;

        $clients[$connId]['conn']->send(json_encode([
            'event'   => 'pdv:pagamento_confirmado',
            'payload' => $data,
            'meta'    => ['ts' => time()],
        ]));

        echo "[PAGTO] → PDV {$pdvKey} venda={$data['numero_venda']}\n";
    });

    /**
     * Watchdog: PDVs que não mandam heartbeat há mais de 90s são declarados offline.
     */
    Timer::add(30, function() use (&$clients, &$pdvIndex) {
        $agora = time();
        foreach ($pdvIndex as $pdvKey => $connId) {
            $client = $clients[$connId] ?? null;
            if (!$client) { unset($pdvIndex[$pdvKey]); continue; }
            if ($agora - $client['last_ping'] > 90) {
                echo "[WATCHDOG] PDV {$pdvKey} sem heartbeat → forçando close\n";
                $client['conn']->close();
            }
        }
    });
};

// ——— Helpers privados ———————————————————————————————————————————————————————

/**
 * Autentica um PDV Electron.
 * Valida token contra o banco (cache Redis 60s).
 */
function _autenticarPdv(
    TcpConnection $conn,
```

```
    array &$client,
    array $payload,
    array &$pdvIndex,
    array &$clients
): void {
    $token    = trim($payload['token']     ?? '');
    $lojaId   = (int) ($payload['loja_id']   ?? 0);
    $numeroPdv = trim($payload['numero_pdv'] ?? '');
    $usuarioId = (int) ($payload['usuario_id'] ?? 0);

    if (!$token || !$lojaId || !$numeroPdv) {
        $conn->send(json_encode(['event' => 'ws:auth_fail', 'payload' => ['motivo' => 'Dados incompletos']]));
        $conn->close();
        return;
    }

    // Valida token (a validação real é feita pelo endpoint PHP;
    // aqui verificamos contra Redis para não abrir PDO no WS)
    $redis = new Redis();
    $redis->connect('127.0.0.1', 6379);

    $tokenCacheKey = "ws:token:{$lojaId}";
    $tokenValido = $redis->get($tokenCacheKey);

    if (!$tokenValido) {
        // Busca no banco e cacheia por 60s
        try {
            $pdo = new PDO('mysql:host=' . DB_HOST . ';dbname=' . DB_NAME, DB_USER, DB_PASS);
            $stmt = $pdo->prepare("SELECT valor FROM config WHERE chave = 'api_token' LIMIT 1");
            $stmt->execute();
            $row = $stmt->fetch(PDO::FETCH_OBJ);
            $tokenValido = $row ? $row->valor : '';
            $redis->setex($tokenCacheKey, 60, $tokenValido);
        } catch (\Throwable $e) {
            $conn->send(json_encode(['event' => 'ws:auth_fail', 'payload' => ['motivo' => 'Erro interno']]));
            $conn->close();
            return;
        }
    }

    if (!hash_equals($tokenValido, $token)) {
        $conn->send(json_encode(['event' => 'ws:auth_fail', 'payload' => ['motivo' => 'Token inválido']]));
        $conn->close();
        return;
    }

    // Cancela timer de timeout de autenticação
```

```
    if (!empty($conn->authTimer)) {
        Timer::del($conn->authTimer);
    }

    $pdvKey = "{$lojaId}:{$numeroPdv}";

    // Desconecta conexão antiga se o mesmo PDV reconectar
    if (isset($pdvIndex[$pdvKey])) {
        $oldConnId = $pdvIndex[$pdvKey];
        if (isset($clients[$oldConnId])) {
            $clients[$oldConnId]['conn']->close();
        }
    }

    $client['tipo']       = 'pdv';
    $client['loja_id']    = $lojaId;
    $client['numero_pdv'] = $numeroPdv;
    $client['usuario_id'] = $usuarioId;
    $client['auth']       = true;
    $client['last_ping']  = time();

    $pdvIndex[$pdvKey] = $conn->id;

    $conn->send(json_encode([
        'event'   => 'ws:auth_ok',
        'payload' => ['numero_pdv' => $numeroPdv, 'loja_id' => $lojaId],
    ]));

    // Notifica admins que PDV está online
    _broadcastAdmin($clients, $lojaId, [
        'event'   => 'admin:pdv_online',
        'payload' => ['loja_id' => $lojaId, 'numero_pdv' => $numeroPdv, 'usuario_id' => $usuarioId],
    ]);

    echo "[AUTH] PDV {$pdvKey} autenticado\n";
}

/**
 * Autentica um cliente admin (painel web).
 * Valida session_id ou JWT (a definir).
 */
function _autenticarAdmin(TcpConnection $conn, array &$client, array $payload): void
{
    // Por ora: valida session_id PHP
    // Em produção: usar JWT assinado gerado no login
    $sessionId = $payload['session_id'] ?? '';
    $lojaId    = (int) ($payload['loja_id'] ?? 0);
```

```
    // TODO: validar session_id via Redis (o AuthController deve gravar após login)
    // session_start(); session_id($sessionId); verificar $_SESSION['logado']

    $client['tipo']    = 'admin';
    $client['loja_id'] = $lojaId;
    $client['auth']    = true;

    $conn->send(json_encode(['event' => 'ws:auth_ok', 'payload' => ['tipo' => 'admin']]));
    echo "[AUTH] Admin conectado loja={$lojaId}\n";
}

/**
 * Envia mensagem para todos os admins de uma loja específica.
 */
function _broadcastAdmin(array &$clients, ?int $lojaId, array $msg): void
{
    $json = json_encode($msg);
    foreach ($clients as $client) {
        if ($client['tipo'] === 'admin' && $client['loja_id'] === $lojaId) {
            $client['conn']->send($json);
        }
    }
}

// ─── Config ──────────────────────────────────────────────────────────────
Worker::$stdoutFile = '/var/www/html/pdvix_ws.log';
Worker::$pidFile    = '/tmp/pdvix_ws.pid';
Worker::$logFile    = '/var/www/html/pdvix_ws_workerman.log';

Worker::runAll();
```

7.6 Como o PDV Electron consome o WebSocket

```javascript
```

```
// pdv_websocket.js — módulo Electron
class PdvWebSocket {
  constructor({ wsUrl, token, lojaId, numeroPdv, usuarioId }) {
    this.wsUrl     = wsUrl;       // ex: wss://seu-servidor.com:8443
    this.token     = token;       // api_token da config local
    this.lojaId    = lojaId;
    this.numeroPdv = numeroPdv;
    this.usuarioId = usuarioId;
    this.ws        = null;
    this.heartbeatInterval = null;
    this.reconnectDelay    = 5000;
  }

  connect() {
    this.ws = new WebSocket(this.wsUrl);

    this.ws.onopen = () => {
      // 1. Autenticar imediatamente
      this.send('pdv:auth', {
        token:      this.token,
        loja_id:    this.lojaId,
        numero_pdv: this.numeroPdv,
        usuario_id: this.usuarioId,
      });
    };

    this.ws.onmessage = (e) => {
      const msg = JSON.parse(e.data);
      this._handleMessage(msg);
    };

    this.ws.onclose = () => {
      this._stopHeartbeat();
      setTimeout(() => this.connect(), this.reconnectDelay);
    };

    this.ws.onerror = (err) => console.error('[WS]', err);
  }

  _handleMessage({ event, payload }) {
    switch (event) {
      case 'ws:auth_ok':
        this._startHeartbeat();
        break;

      case 'ws:auth_fail':
```

```
      console.error('[WS] Auth falhou:', payload.motivo);
      break;

    case 'pdv:comando':
      this._executarComando(payload);
      break;

    case 'pdv:pagamento_confirmado':
      this._onPagamentoConfirmado(payload);
      break;

    case 'pdv:pagamento_cancelado':
      this._onPagamentoCancelado(payload);
      break;

    case 'pdv:carga_disponivel':
      // Baixar nova carga automaticamente
      sincronizarCargaInicial();
      break;
  }
}

_executarComando({ cmd_id, tipo, dados }) {
  const resultado = { cmd_id, sucesso: true, mensagem: 'OK' };

  try {
    switch (tipo) {
      case 'reiniciar':        app.relaunch(); app.exit(); break;
      case 'desligar':         app.exit(); break;
      case 'fechar_caixa':     fecharCaixaForcado(dados); break;
      case 'enviar_carga':     sincronizarCargaInicial(); break;
      case 'cancelar_item':    cancelarItemVenda(dados); break;
      case 'cancelar_venda':   cancelarVenda(dados); break;
      case 'desconto_item':    aplicarDescontoItem(dados); break;
      case 'desconto_venda':   aplicarDescontoVenda(dados); break;
      case 'finalizar_venda':  finalizarVendaRemota(dados); break;
      case 'enviar_comanda':   receberComanda(dados); break;
      default: resultado.sucesso = false; resultado.mensagem = `Comando desconhecido: ${tipo}`;
    }
  } catch (e) {
    resultado.sucesso  = false;
    resultado.mensagem = e.message;
  }

  this.send('pdv:cmd_resultado', resultado);
}
```

```
  _onPagamentoConfirmado(payload) {
    // Atualiza pagamento na venda aberta no SQLite local
    db.run(
      `UPDATE pagamentos_venda SET status = 'confirmado' WHERE venda_id =
       (SELECT id FROM vendas WHERE numero_venda = ?)`,
      [payload.numero_venda]
    );
    // Dispara evento para UI do PDV
    mainWindow.webContents.send('pagamento:confirmado', payload);
  }

  send(event, payload = {}) {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify({ event, payload, meta: { ts: Date.now() } }));
    }
  }

  _startHeartbeat() {
    this.heartbeatInterval = setInterval(() => {
      this.send('pdv:heartbeat', {
        loja_id:        this.lojaId,
        numero_pdv:     this.numeroPdv,
        status:         'online',
        caixa_aberto:   getCaixaStatus(),
        ultima_venda_em: getUltimaVendaEm(),
      });
    }, 30000); // 30s
  }

  _stopHeartbeat() {
    clearInterval(this.heartbeatInterval);
  }
}
```

---

8. Novos Módulos — Design e Especificação

8.1 Multi-Lojas

Tabela `lojas` (nova)

```sql
```

```sql
CREATE TABLE lojas (
  id          INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  nome        VARCHAR(100) NOT NULL,
  cnpj        VARCHAR(18) NOT NULL UNIQUE,
  endereco    TEXT NOT NULL,
  numero      VARCHAR(20),
  bairro      VARCHAR(80),
  cidade      VARCHAR(80),
  estado      CHAR(2),
  cep         VARCHAR(10),
  telefone    VARCHAR(20),
  status      ENUM('ativa','inativa') NOT NULL DEFAULT 'ativa',
  created_at  DATETIME DEFAULT NOW(),
  updated_at  DATETIME DEFAULT NOW() ON UPDATE NOW()
);
```

Tabela `loja_usuarios` (nova — relacionamento N:N)

sql

```sql
CREATE TABLE loja_usuarios (
  loja_id    INT NOT NULL,
  usuario_id INT NOT NULL,
  PRIMARY KEY (loja_id, usuario_id),
  FOREIGN KEY (loja_id)    REFERENCES lojas (id)    ON DELETE CASCADE,
  FOREIGN KEY (usuario_id) REFERENCES usuarios (id) ON DELETE CASCADE
);
```

Tabela `pdvs` (nova)

sql

```sql
CREATE TABLE pdvs (
  id          INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  loja_id     INT NOT NULL,
  numero_pdv  VARCHAR(10) NOT NULL,       -- ex: '01', '02'
  descricao   VARCHAR(100),               -- ex: 'PDV Caixa 1'
  url_local   VARCHAR(200),               -- ex: http://192.168.1.50:3000
  token_local VARCHAR(100),               -- token alternativo por PDV (futuro)
  status      ENUM('ativo','inativo') NOT NULL DEFAULT 'ativo',
  ultimo_ping DATETIME,                   -- atualizado pelo WebSocket heartbeat
  online      TINYINT DEFAULT 0,          -- 1 = WS conectado
  versao_app  VARCHAR(20),                -- ex: '2.1.0'
  created_at  DATETIME DEFAULT NOW(),
  UNIQUE KEY uk_loja_pdv (loja_id, numero_pdv),
  FOREIGN KEY (loja_id) REFERENCES lojas (id) ON DELETE CASCADE
);
```

Colunas `loja_id` a adicionar (migration v3)

```sql
ALTER TABLE produtos       ADD COLUMN loja_id INT AFTER id, ADD FOREIGN KEY (loja_id) REFERENCES lojas(
ALTER TABLE estoque        ADD COLUMN loja_id INT AFTER id, ADD FOREIGN KEY (loja_id) REFERENCES lojas(
ALTER TABLE vendas         ADD COLUMN loja_id INT AFTER id, ADD FOREIGN KEY (loja_id) REFERENCES lojas(
ALTER TABLE caixa_sessoes  ADD COLUMN loja_id INT AFTER id, ADD FOREIGN KEY (loja_id) REFERENCES loja
ALTER TABLE supervisores_cartoes ADD COLUMN loja_id INT AFTER id, ADD FOREIGN KEY (loja_id) REFERENCE
```

API REST `/api/lojas`

Método	Rota	Perfil	Descrição
GET	`/api/lojas`	administrador	Listar lojas
POST	`/api/lojas`	administrador	Criar loja
PUT	`/api/lojas`	administrador	Atualizar loja
DELETE	`/api/lojas`	administrador	Inativar loja
GET	`/api/lojas/pdvs?loja_id=X`	gerente	Listar PDVs da loja
POST	`/api/lojas/pdvs`	administrador	Cadastrar PDV
PUT	`/api/lojas/pdvs`	administrador	Atualizar PDV

8.2 Módulo PDV (Configuração e Comandos Remotos)

API REST `/api/pdv/`

Método	Rota	Auth	Descrição
GET	`/api/pdv/status`	sessão	Status de todos os PDVs (online/offline, último ping)
GET	`/api/pdv/status?loja_id=X`	sessão	Status por loja
POST	`/api/pdv/comando`	sessão	Enfileira comando remoto via Redis
POST	`/api/pdv/carga`	sessão	Sinaliza carga disponível (enfileira no Redis)

`POST /api/pdv/comando`

```json
{
  "loja_id":    1,
  "numero_pdv": "01",
  "tipo":       "cancelar_item",
  "dados": {
    "venda_id_local": 123,
    "produto_id":     5,
    "quantidade":     2
  }
}
```

O controller:

1. Valida sessão e perfil (`gerente`+)
2. Verifica se PDV está online (tabela `pdvs.online = 1`)
3. Enfileira no Redis: `RPUSH pdv:cmd:{loja_id}:{numero_pdv} {json}`
4. Retorna `202 Accepted` — resultado chega via WebSocket

`POST /api/pdv/carga`

```json
{ "loja_id": 1, "numero_pdv": "01", "tipo": "completa" }
```

Enfileira: `RPUSH pdv:cmd:{loja_id}:{numero_pdv} {"event":"pdv:carga_disponivel","payload":{"tipo":"completa"}}`

Front-end `pdv.html/js` — Tela de Gerenciamento de PDVs

Cards por PDV:

- Badge Online/Offline (atualizado via WebSocket)
- Último ping (tempo relativo)
- Operador logado
- Caixa: aberto/fechado
- Botões: Enviar carga, Comandos, Ver detalhes

Comandos disponíveis no modal:

- Reiniciar PDV
- Fechar caixa (com campo de motivo)
- Cancelar item (campo: número da venda + produto)
- Cancelar venda (campo: número da venda)
- Desconto em item/venda
- Enviar comanda

8.3 Pagar.me — PIX e Stone POS

Fluxo PIX

```
1. PDV solicita PIX via WS:    pdv:solicitar_pix { numero_venda, valor }
   OU via REST:                POST /api/pagamentos/pix/criar

2. Servidor chama pagar.me:
   POST https://api.pagar.me/core/v5/orders
   { customer, items, payments: [{ payment_method: 'pix', pix: { expires_in: 300 } }] }

3. Servidor salva na fila:
   RPUSH pdv:pagamento_pendente:{numero_venda}  { qr_code, qr_code_url, charge_id, order_id }
   SET   pdv:venda_pdv:{numero_venda}  "{loja_id}:{numero_pdv}"  EX 600

4. WS distribui QR Code para o PDV via evento pdv:pagamento_pendente

5. PDV exibe QR Code na tela

6. Pagar.me chama webhook:
   POST /api/webhook/pagarme

7. WebhookController valida assinatura X-Hub-Signature
   Atualiza pagamentos_venda SET status = 'confirmado', referencia_externa = order_id
   RPUSH pdv:fila_pagamentos { numero_venda, valor, tipo: 'pix', referencia }

8. WS lê fila e envia pdv:pagamento_confirmado para o PDV correto

9. PDV finaliza a venda no SQLite local e dispara sync
```

Tabela `pagarme_transacoes` (nova)

```sql
```

```sql
CREATE TABLE pagarme_transacoes (
  id            INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  venda_id       INT NOT NULL,
  loja_id        INT NOT NULL,
  order_id       VARCHAR(100) NOT NULL UNIQUE,  -- id do pagar.me
  charge_id      VARCHAR(100),
  tipo           ENUM('pix','pos_debito','pos_credito','pos_pix') NOT NULL,
  valor          DECIMAL(10,2) NOT NULL,
  status         ENUM('pending','paid','failed','canceled','processing') DEFAULT 'pending',
  qr_code        TEXT,
  qr_code_url     TEXT,
  expires_at      DATETIME,
  payload_request  JSON,
  payload_response JSON,
  payload_webhook  JSON,
  created_at      DATETIME DEFAULT NOW(),
  updated_at      DATETIME DEFAULT NOW() ON UPDATE NOW(),
  FOREIGN KEY (venda_id) REFERENCES vendas (id),
  FOREIGN KEY (loja_id)  REFERENCES lojas  (id)
);
```

API REST pagar.me

Método	Rota	Auth	Descrição
POST	`/api/pagamentos/pix/criar`	sessão/token	Cria order PIX no pagar.me
POST	`/api/pagamentos/pix/cancelar`	sessão	Cancela charge PIX pendente
POST	`/api/webhook/pagarme`	Assinatura HMAC	Recebe eventos do pagar.me
GET	`/api/pagamentos/pix/status?order_id=X`	sessão	Consulta status de uma charge

Webhook Pagar.me — `POST /api/webhook/pagarme`

```php
```

```
// Validação obrigatória:
$assinatura = $_SERVER['HTTP_X_HUB_SIGNATURE'] ?? '';
$corpo      = file_get_contents('php://input');
$segredo    = getenv('PAGARME_WEBHOOK_SECRET');
$esperado   = 'sha1=' . hash_hmac('sha1', $corpo, $segredo);

if (!hash_equals($esperado, $assinatura)) {
   http_response_code(401); exit;
}
```

Regras de pagamento (conforme especificação)

1. **Uma venda pode ter no máximo 1 pagamento `pendente` por vez**
   - Se existe `pendente`, o PDV/admin deve cancelar antes de criar outro
2. **Pagamentos `pix`, `pos_debito` e `pos_credito` passam pelo pagar.me**
3. **`dinheiro`, `pos_pix`, `convenio`, `outros` são registrados diretamente** (sem integração)
4. **O PDV recebe confirmação via WebSocket** — não precisa fazer polling

---

8.4 Dashboard de Vendas

API REST `/api/dashboard`

Método	Rota	Auth	Descrição
GET	`/api/dashboard/resumo`	gerente	Cards: vendas hoje, ticket médio, cancelamentos, receita
GET	`/api/dashboard/vendas-hora`	gerente	Vendas por hora (gráfico de linha)
GET	`/api/dashboard/top-produtos`	gerente	Top 10 produtos mais vendidos
GET	`/api/dashboard/formas-pagamento`	gerente	Receita por forma de pagamento (pizza)
GET	`/api/dashboard/pdvs-status`	gerente	Status de cada PDV

Filtros comuns: `loja_id`, `data_inicio`, `data_fim`

8.5 Cancelamentos

Tabela `cancelamentos` (nova)

```sql
```

```sql
CREATE TABLE cancelamentos (
  id            INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  tipo          ENUM('venda','item') NOT NULL,
  venda_id      INT NOT NULL,
  venda_item_id INT,            -- NULL se tipo='venda'
  loja_id       INT NOT NULL,
  numero_pdv    VARCHAR(10),
  usuario_id    INT NOT NULL,    -- quem cancelou
  supervisor_id INT,            -- quem autorizou (cartão supervisor)
  motivo        VARCHAR(200),
  valor_cancelado DECIMAL(10,2) NOT NULL,
  cancelado_em  DATETIME NOT NULL DEFAULT NOW(),
  origem        ENUM('pdv','painel') NOT NULL DEFAULT 'pdv',
  FOREIGN KEY (venda_id) REFERENCES vendas (id),
  FOREIGN KEY (loja_id)  REFERENCES lojas  (id)
);
```

API REST `/api/cancelamentos`

Método	Rota	Auth	Descrição
GET	`/api/cancelamentos`	gerente	Listar cancelamentos (DataTables)
POST	`/api/cancelamentos/venda`	gerente	Cancelar venda via painel
POST	`/api/cancelamentos/item`	gerente	Cancelar item via painel
POST	`/api/pdv/sync-cancelamento`	token	PDV sincroniza cancelamentos offline

Fluxo de cancelamento no PDV

1. Operador solicita cancelamento (requer cartão supervisor se configurado)
2. PDV registra no SQLite: INSERT INTO cancelamentos (local)
3. PDV estorna estoque localmente
4. Ao sincronizar: POST /api/pdv/sync-cancelamento
5. Servidor: estorna estoque no MariaDB + registra em cancelamentos

---

8.6 Espelho PDV

Relatório que importa um log exportado pelo PDV via API autenticada.

API REST

Método	Rota	Auth	Descrição
GET	`/api/espelho-pdv?caixa_sessao_id=X`	token ou sessão	Retorna espelho completo da sessão

Estrutura da resposta:

```json
{
  "caixa": { ...dados_sessao... },
  "vendas": [
    {
      "numero_venda": "PDV01-xxx",
      "data_venda":   "...",
      "total":        78.00,
      "status":       "finalizada",
      "itens": [...],
      "pagamentos": [...]
    }
  ],
  "sangrias": [...],
  "cancelamentos": [...],
  "resumo": {
    "total_bruto":    780.00,
    "total_desconto": 10.00,
    "total_liquido":  770.00,
    "por_forma": { "dinheiro": 500.00, "pix": 270.00 }
  }
}
```

Front-end: Página `/espelho-caixa?id=X` — renderização HTML para impressão.

8.7 Comandas (Pré-Venda)

Tabela `comandas` (nova)

```sql
```

```sql
CREATE TABLE comandas (
  id          INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  loja_id     INT NOT NULL,
  numero      VARCHAR(30) NOT NULL UNIQUE,   -- ex: CMD-0001
  cliente_id  INT,
  cliente_nome VARCHAR(160) DEFAULT 'CONSUMIDOR FINAL',
  usuario_id  INT NOT NULL,                  -- quem criou
  pdv_destino VARCHAR(10),                   -- número do PDV para enviar
  status      ENUM('aberta','enviada','finalizada','cancelada') DEFAULT 'aberta',
  observacao  TEXT,
  created_at  DATETIME DEFAULT NOW(),
  updated_at  DATETIME DEFAULT NOW() ON UPDATE NOW(),
  FOREIGN KEY (loja_id) REFERENCES lojas (id)
);
```

```sql
CREATE TABLE comanda_itens (
  id           INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  comanda_id   INT NOT NULL,
  produto_id   INT NOT NULL,
  quantidade   DECIMAL(10,3) NOT NULL,
  valor_unitario DECIMAL(10,2) NOT NULL,
  subtotal     DECIMAL(10,2) NOT NULL,
  observacao   TEXT,
  FOREIGN KEY (comanda_id) REFERENCES comandas (id) ON DELETE CASCADE,
  FOREIGN KEY (produto_id) REFERENCES produtos (id)
);
```

API REST `/api/comandas`

Método	Rota	Auth	Descrição
GET	`/api/comandas`	operador	Listar (DataTables)
POST	`/api/comandas`	operador	Criar comanda
PUT	`/api/comandas`	operador	Atualizar comanda/itens
DELETE	`/api/comandas`	gerente	Cancelar comanda
POST	`/api/comandas/enviar`	gerente	Envia comanda para PDV via WebSocket

Fluxo de envio para PDV

```
1. Admin cria comanda com itens no painel
2. Admin escolhe "Enviar para PDV" → seleciona loja + número do PDV
3. API enfileira no Redis: RPUSH pdv:cmd:{loja_id}:{numero_pdv} enviar_comanda
4. WS entrega para o PDV
5. PDV cria venda localmente com os itens da comanda
6. Operador finaliza normalmente no PDV
7. PDV sincroniza venda com número de comanda no campo observacao
```

---

9. Migrations Necessárias

Migration v3 — Multi-loja + PDVs

```sql
```

```sql
-- migration_v3.sql
-- Execute em ordem. Faça backup antes.

-- 1. Lojas
CREATE TABLE IF NOT EXISTS lojas (
  id        INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  nome      VARCHAR(100) NOT NULL,
  cnpj      VARCHAR(18) NOT NULL UNIQUE,
  endereco  TEXT NOT NULL,
  numero    VARCHAR(20),
  bairro    VARCHAR(80),
  cidade    VARCHAR(80),
  estado    CHAR(2),
  cep       VARCHAR(10),
  telefone  VARCHAR(20),
  status    ENUM('ativa','inativa') NOT NULL DEFAULT 'ativa',
  created_at DATETIME DEFAULT NOW(),
  updated_at DATETIME DEFAULT NOW() ON UPDATE NOW()
);

-- Loja padrão para migração dos dados existentes
INSERT INTO lojas (nome, cnpj, endereco, status)
VALUES ('Loja Principal', '00000000000000', 'Endereço não cadastrado', 'ativa');

-- 2. PDVs
CREATE TABLE IF NOT EXISTS pdvs (
  id        INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  loja_id   INT NOT NULL,
  numero_pdv  VARCHAR(10) NOT NULL,
  descricao   VARCHAR(100),
  url_local   VARCHAR(200),
  status      ENUM('ativo','inativo') NOT NULL DEFAULT 'ativo',
  ultimo_ping DATETIME,
  online      TINYINT DEFAULT 0,
  versao_app  VARCHAR(20),
  created_at  DATETIME DEFAULT NOW(),
  UNIQUE KEY uk_loja_pdv (loja_id, numero_pdv),
  FOREIGN KEY (loja_id) REFERENCES lojas (id) ON DELETE CASCADE
);

-- PDV padrão
INSERT INTO pdvs (loja_id, numero_pdv, descricao)
SELECT 1, '01', 'PDV Principal' FROM DUAL
WHERE NOT EXISTS (SELECT 1 FROM pdvs WHERE loja_id = 1 AND numero_pdv = '01');

-- 3. Relacionamento usuário × loja
```

```sql
CREATE TABLE IF NOT EXISTS loja_usuarios (
  loja_id    INT NOT NULL,
  usuario_id INT NOT NULL,
  PRIMARY KEY (loja_id, usuario_id),
  FOREIGN KEY (loja_id)    REFERENCES lojas    (id) ON DELETE CASCADE,
  FOREIGN KEY (usuario_id) REFERENCES usuarios (id) ON DELETE CASCADE
);

-- Associa todos os usuários existentes à loja 1
INSERT IGNORE INTO loja_usuarios (loja_id, usuario_id)
SELECT 1, id FROM usuarios;

-- 4. Adiciona loja_id nas tabelas principais
ALTER TABLE vendas       ADD COLUMN loja_id INT DEFAULT 1 AFTER id;
ALTER TABLE caixa_sessoes ADD COLUMN loja_id INT DEFAULT 1 AFTER id;
ALTER TABLE produtos      ADD COLUMN loja_id INT DEFAULT 1 AFTER id;
ALTER TABLE estoque       ADD COLUMN loja_id INT DEFAULT 1 AFTER produto_id;
ALTER TABLE supervisores_cartoes ADD COLUMN loja_id INT DEFAULT 1 AFTER id;

-- FKs após preencher os dados
ALTER TABLE vendas       ADD CONSTRAINT fk_venda_loja     FOREIGN KEY (loja_id) REFERENCES lojas(id);
ALTER TABLE caixa_sessoes ADD CONSTRAINT fk_caixa_loja     FOREIGN KEY (loja_id) REFERENCES lojas(id);
ALTER TABLE produtos      ADD CONSTRAINT fk_produto_loja   FOREIGN KEY (loja_id) REFERENCES lojas(id);
ALTER TABLE supervisores_cartoes ADD CONSTRAINT fk_supcard_loja FOREIGN KEY (loja_id) REFERENCES lojas(

-- 5. Cancelamentos
CREATE TABLE IF NOT EXISTS cancelamentos (
  id             INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  tipo           ENUM('venda','item') NOT NULL,
  venda_id       INT NOT NULL,
  venda_item_id  INT,
  loja_id        INT NOT NULL,
  numero_pdv     VARCHAR(10),
  usuario_id     INT NOT NULL,
  supervisor_id  INT,
  motivo         VARCHAR(200),
  valor_cancelado DECIMAL(10,2) NOT NULL,
  cancelado_em   DATETIME NOT NULL DEFAULT NOW(),
  origem         ENUM('pdv','painel') NOT NULL DEFAULT 'pdv',
  FOREIGN KEY (venda_id) REFERENCES vendas (id),
  FOREIGN KEY (loja_id)  REFERENCES lojas  (id)
);

-- 6. Pagar.me
CREATE TABLE IF NOT EXISTS pagarme_transacoes (
  id             INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  venda_id       INT NOT NULL,
```

```sql
  loja_id         INT NOT NULL,
  order_id        VARCHAR(100) NOT NULL UNIQUE,
  charge_id       VARCHAR(100),
  tipo            ENUM('pix','pos_debito','pos_credito','pos_pix') NOT NULL,
  valor           DECIMAL(10,2) NOT NULL,
  status          ENUM('pending','paid','failed','canceled','processing') DEFAULT 'pending',
  qr_code         TEXT,
  qr_code_url     TEXT,
  expires_at      DATETIME,
  payload_request  JSON,
  payload_response JSON,
  payload_webhook  JSON,
  created_at      DATETIME DEFAULT NOW(),
  updated_at      DATETIME DEFAULT NOW() ON UPDATE NOW(),
  FOREIGN KEY (venda_id) REFERENCES vendas (id),
  FOREIGN KEY (loja_id)  REFERENCES lojas  (id)
);

-- Coluna para vincular pagamento ao charge do pagar.me
ALTER TABLE pagamentos_venda
  ADD COLUMN pagarme_order_id  VARCHAR(100) DEFAULT NULL AFTER referencia_externa,
  ADD COLUMN pagarme_charge_id VARCHAR(100) DEFAULT NULL AFTER pagarme_order_id;

-- 7. Comandas
CREATE TABLE IF NOT EXISTS comandas (
  id          INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  loja_id     INT NOT NULL,
  numero      VARCHAR(30) NOT NULL UNIQUE,
  cliente_id  INT,
  cliente_nome VARCHAR(160) DEFAULT 'CONSUMIDOR FINAL',
  usuario_id  INT NOT NULL,
  pdv_destino  VARCHAR(10),
  status      ENUM('aberta','enviada','finalizada','cancelada') DEFAULT 'aberta',
  observacao  TEXT,
  created_at  DATETIME DEFAULT NOW(),
  updated_at  DATETIME DEFAULT NOW() ON UPDATE NOW(),
  FOREIGN KEY (loja_id)    REFERENCES lojas    (id),
  FOREIGN KEY (usuario_id) REFERENCES usuarios (id)
);

CREATE TABLE IF NOT EXISTS comanda_itens (
  id           INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  comanda_id   INT NOT NULL,
  produto_id   INT NOT NULL,
  quantidade   DECIMAL(10,3) NOT NULL,
  valor_unitario DECIMAL(10,2) NOT NULL,
  subtotal     DECIMAL(10,2) NOT NULL,
```

```sql
  observacao     TEXT,
  FOREIGN KEY (comanda_id) REFERENCES comandas (id) ON DELETE CASCADE,
  FOREIGN KEY (produto_id) REFERENCES produtos (id)
);

-- 8. Índices de performance
CREATE INDEX idx_vendas_loja_data       ON vendas       (loja_id, data_venda);
CREATE INDEX idx_caixa_loja_abertura    ON caixa_sessoes (loja_id, abertura_em);
CREATE INDEX idx_pagarme_status         ON pagarme_transacoes (status, created_at);
CREATE INDEX idx_cancelamentos_loja_data ON cancelamentos (loja_id, cancelado_em);
CREATE INDEX idx_vendas_numero          ON vendas       (numero_venda);

-- 9. numero_venda no sync precisa do loja_id
ALTER TABLE vendas ADD COLUMN UNIQUE KEY uk_numero_venda (numero_venda);
-- (já existe como chave, confirmar antes)
```

---

10. Pendências no Banco SQLite do PDV

> ⚠️ **Não alterar agora.** Registrar para sincronizar depois.

#	Alteração	Tabela SQLite	Motivo
1	Adicionar `loja_id INT DEFAULT 1`	`vendas`, `caixa_sessoes`	Multi-loja — PDV precisa saber qual loja pertence
2	Adicionar `numero_pdv TEXT DEFAULT '01'`	`vendas`	Já existe no servidor mas falta no payload de sync
3	Criar tabela `cancelamentos` local	—	Cancelamentos offline precisam de controle de sync
4	Adicionar `sincronizado INTEGER DEFAULT 0`	`cancelamentos`	Controle de sync igual às vendas
5	Adicionar `pagarme_order_id TEXT`	`pagamentos_venda`	Guardar order_id do PIX gerado
6	Criar tabela `comandas` e `comanda_itens` local	—	Receber pré-vendas do admin
7	Adicionar `desconto_item REAL DEFAULT 0`	`venda_itens`	Já existe no SQLite ✅
8	Criar tabela `pdv_logs`	—	Espelho PDV — log de todas as operações

---

11. Checklist de Implementação

Fase 1 — Base (sem essa nada funciona)

- [ ] Executar `migration_v3.sql`
- [ ] Criar `LojaController` + `LojaModel`
- [ ] Criar `PdvController` (config + comandos)
- [ ] Criar `pdv_server.php` (WebSocket)
- [ ] Registrar rotas novas em `routes.php`
- [ ] Criar `lojas.html/js` e `pdv.html/js`
- [ ] Atualizar `CargaInicialController` para incluir `loja_id`
- [ ] Atualizar `PdvSyncController` para receber e gravar `loja_id`

Fase 2 — Pagamentos

- [ ] Criar `PagarmeService` (HTTP client para API pagar.me)
- [ ] Criar `PagarmeController` com `/api/pagamentos/pix/criar` e `/api/webhook/pagarme`
- [ ] Criar tabela `pagarme_transacoes`
- [ ] Integrar webhook → Redis → WebSocket → PDV
- [ ] Front-end: botão PIX no painel de vendas
- [ ] PDV: exibir QR Code ao receber `pdv:pagamento_pendente`

Fase 3 — Cancelamentos

- [ ] Criar `CancelamentoController` + `CancelamentoModel`
- [ ] Criar rota `POST /api/pdv/sync-cancelamento`
- [ ] Adicionar tabela `cancelamentos` (SQLite + MariaDB)
- [ ] Front-end: `cancelamentos.html/js`
- [ ] PDV: criar tabela local + sync de cancelamentos

Fase 4 — Comandas

- [ ] Criar `ComandaController` + `ComandaModel`
- [ ] Criar tabelas `comandas` + `comanda_itens`
- [ ] Front-end: `comandas.html/js`
- [ ] PDV: receber comanda via WS e criar venda local

Fase 5 — Dashboard e Relatórios

- [ ] Criar `DashboardController`
- [ ] Front-end: `dashboard.html/js` com gráficos
- [ ] Criar endpoint `/api/espelho-pdv`
- [ ] Página de impressão `/espelho-caixa`

Fase 6 — Refinamentos

- [ ] Refatorar `requirePerfil()` para trait (`App\Core\Traits\AuthorizaTrait`)
- [ ] Adicionar `loja_id` a todos os filtros DataTables
- [ ] Isolamento de dados por loja no painel
- [ ] Testes de carga e revisão de índices

Documento gerado com base na análise completa de: Controllers, Models, Services, Routes, Schema MariaDB, Schema SQLite, WebSocket server de referência e padrões de front-end (SPA AdminLTE).